School of Future Tech

Case Study Report

on

TutorBridge: One-on-One Tutoring Portal

by

Kaushal Rajmandai
150096725111

Index

1. Introduction to the Case Study
2. Problem Statement / Case Background (Abstract)
3. Problem Statement / Case Study Design
4. Methods & Algorithms / Technology Applied
5. Implementation Details and Snapshots
6. Results and Conclusion
7. References

1. Introduction to the Case Study

The internet has transformed education, making it possible for students and tutors to connect from anywhere in the world. However, students still face challenges finding qualified, subject-specific tutors for live one-on-one sessions. TutorBridge is a web-based tutoring portal designed to solve this problem by providing a structured, responsive platform where students can browse verified tutors, view qualifications and ratings, check availability, and book sessions through an interactive calendar interface.

This case study documents the design and development of the TutorBridge platform using HTML5, CSS3, and JavaScript. The project demonstrates how structured UI design, semantic markup, responsive layouts, and functional JavaScript logic can be combined to build a complete, real-world web application. The system consists of five pages — Home, Tutors, Tutor Details, Booking, and Pricing — each following consistent design principles and sharing a common navigation bar and footer.

The platform targets students at all academic levels who require personalised academic support, and tutors who wish to offer their expertise online. Key features include tutor profile cards with star ratings, qualification badges, hourly rate display, an interactive session booking calendar, and booking confirmation logic.

2. Problem Statement / Case Background (Abstract)

Background

Students often struggle to find subject-specific tutors who match their schedule, budget, and learning style. Existing platforms can be expensive, lack transparency, or provide a poor user experience. There is a clear need for a clean, functional, and accessible platform that puts the student first — allowing them to browse, evaluate, and book tutors with minimal friction.

Abstract

TutorBridge is a fully responsive, multi-page web application that connects students with subject-specific tutors for live online sessions. The platform was built using HTML5 (semantic tags), CSS3 (Flexbox, Grid, animations, keyframes), and vanilla JavaScript (DOM manipulation, event handling, date logic). Students can browse tutor cards with photo placeholders, subject expertise, hourly rates, and star ratings. A detailed tutor profile page shows qualifications, badges, experience, a weekly availability calendar, and student reviews. The booking page features an interactive monthly calendar that prevents selection of past dates and already-booked time slots, dynamically displays the selected tutor name, subject, date, time, and total session cost in a live booking summary sidebar, and shows a confirmation message upon successful booking. The pricing page compares pay-per-session plans and prepaid hour bundles with a feature comparison table.

3. Problem Statement / Case Study Design

The TutorBridge platform is designed as a multi-page website with a consistent visual identity and shared components. The design follows a professional education-themed colour palette — deep navy (#0d1b2a), teal (#1b9aaa), and warm cream (#fdf8f2) — with Playfair Display used for headings and DM Sans for body text.

Website Structure (5 Pages)

Home (index.html)

Platform overview, hero banner, stats bar, subject category grid, featured tutor cards, how-it-works section, and call-to-action banner.

Tutors (tutors.html)

Full tutor listing using CSS Grid layout with cards showing photo placeholder, subject expertise, hourly rate, star rating, and qualification badges. Includes a live filter bar for subject, rate, and name search.

Tutor Details (details.html)

Full tutor profile loaded dynamically via URL query parameter (?id=). Displays qualifications, expertise badges, experience, biography, weekly availability calendar, and student reviews.

Booking (booking.html)

Session booking interface with tutor selector, student details form, interactive monthly calendar with past-date blocking, time slot grid with booked-slot simulation, live booking summary sidebar, and booking confirmation display.

Pricing (pricing.html)

Pay-per-session plans (Basic, Standard, Premium) and prepaid hour bundles (Starter, Growth, Pro) displayed in card layout, with a full feature comparison table.

All five pages share a sticky navigation bar with the TutorBridge logo and links, and a consistent footer with quick links, subject categories, and contact details.

4. Methods & Algorithms / Technology Applied

HTML5 Techniques

- Semantic tags used throughout: **<header>**, **<nav>**, **<section>**, **<article>**, **<footer>**.
- Tutor cards built with structured markup: image placeholder, subject name, qualification badge, hourly rate, and star rating section.
- Booking form uses **<form>**, **<input>** (text, email), **<select>**, and **<textarea>** with appropriate labels.
- URL query parameters (**?id=**) used for passing tutor context between pages.

CSS3 Techniques

- Flexbox used for navigation bar layout, hero button groups, footer grid, and card footers.
- CSS Grid used for the tutor listing section, subject category grid, and time slot layout.
- Star rating display implemented using styled Unicode characters with gold colour.
- Box-shadow, border-radius, and spacing techniques applied consistently across cards.
- Sticky navigation bar using **position: sticky; top: 0;**.
- Hover effects on tutor cards (translateY transform), buttons (box-shadow), and subject cards.
- Fade-in animation using **@keyframes fadeInUp** with staggered animation-delay classes.
- Smooth scrolling via **html { scroll-behavior: smooth; }**.
- Input focus styling with teal border-color and rgba box-shadow glow.
- CSS variables (custom properties) for consistent colour palette across all pages.

JavaScript Functional Logic

- **querySelector / getElementById** used for all DOM element access.
- **addEventListener** used for form submission, filter input changes, date selection, and time slot clicks.
- Tutor data stored as a JavaScript array of objects; dynamically rendered to DOM using **innerHTML** and **map()**.
- URL query parameters parsed using **URLSearchParams** to load tutor-specific data on details and booking pages.
- Calendar built programmatically: calculates first day of month, iterates over days, marks past dates as disabled.

• Date comparison logic prevents selection of dates before today.

• Booked time slots stored in an object keyed by date string; slots matching booked entries are marked as unavailable.

• Booking summary sidebar updates dynamically on tutor change, date selection, and time selection.

• Confirmation message displayed via **style.display = "block"** after form validation and booking.

• Conditional statements used for filter logic, past-date checking, and booked-slot detection.

Technology Stack

Language	HTML5, CSS3, Vanilla JavaScript (ES6+)
Fonts	Google Fonts — Playfair Display (headings), DM Sans (body)
Layout	CSS Flexbox and CSS Grid
Animations	CSS @keyframes (fadeInUp), transition properties
Structure	Multi-page website with shared external CSS and JS files
Tools	VS Code, Live Server, Chrome DevTools, GitHub

5. Implementation Details and Snapshots

File Structure

The project follows a clean folder structure with external CSS and JavaScript files as required:

```
tutorbridge/
■■■ index.html — Home page
■■■ tutors.html — Tutor listing page
■■■ details.html — Tutor profile page
■■■ booking.html — Session booking page
■■■ pricing.html — Pricing plans page
■■■ css/
■ ■■■ style.css — All styles (Flexbox, Grid, animations)
■■■ js/
■■■ main.js — All JavaScript logic
```

Key Implementation Details

Tutor Data Model

All tutor data is stored as a JavaScript array of objects in main.js, each containing id, name, subject, expertise array, rate, rating, reviews, experience, qualification, badges, availability (days array), emoji, and bio. This single source of truth is used to render tutor cards, the details page, and the booking dropdown.

Dynamic Page Rendering

Tutor cards on the home page and tutors page are generated dynamically using JavaScript template literals and innerHTML. The details page reads a ?id= query parameter from the URL to identify which tutor to display, then populates all DOM elements accordingly.

Booking Calendar Logic

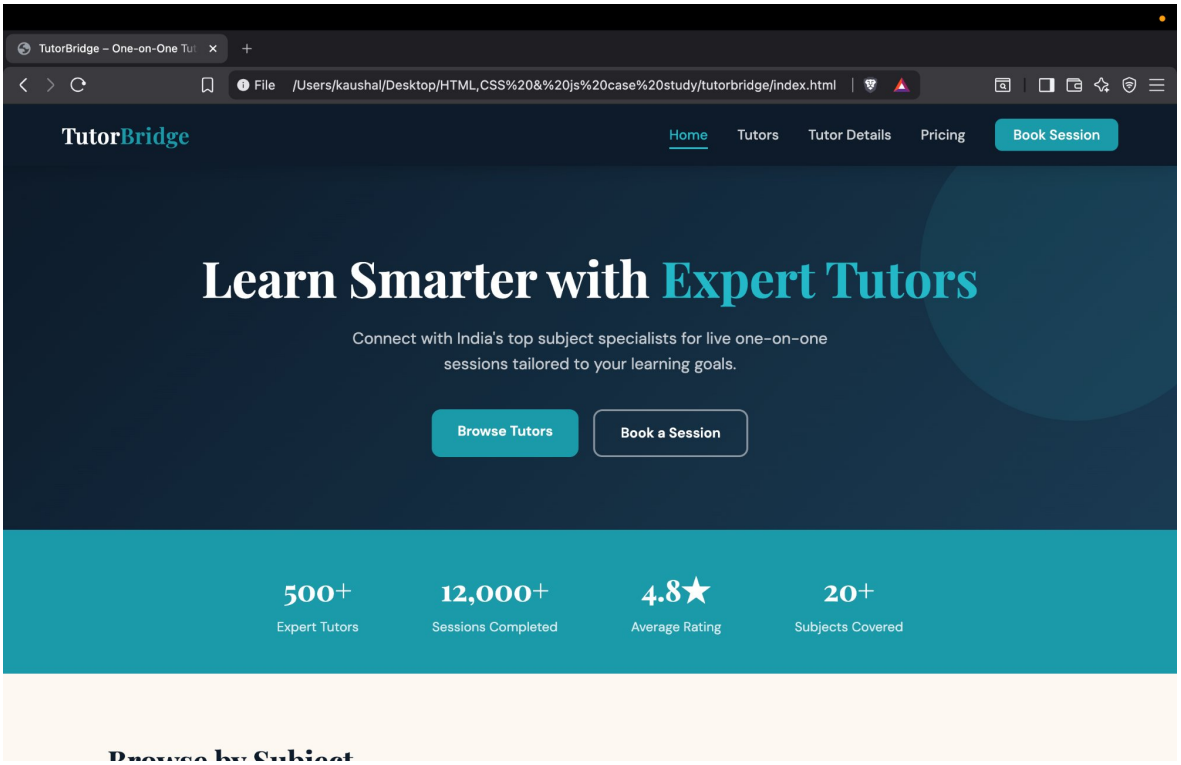
The calendar is built by calculating the first weekday of the current month, then generating numbered day cells. Each cell checks if the day is before today — if so, it receives a "past" class and becomes unclickable. When a date is selected, the time slot grid renders 9 available slots and marks any pre-booked slots (from a simulated bookedSlots object) as disabled.

Filter System

The tutors page filter bar has three inputs — text search, subject dropdown, and max rate dropdown. All three attach change/input event listeners that call a shared applyFilters() function, which filters the tutors array and re-renders the grid in real time.

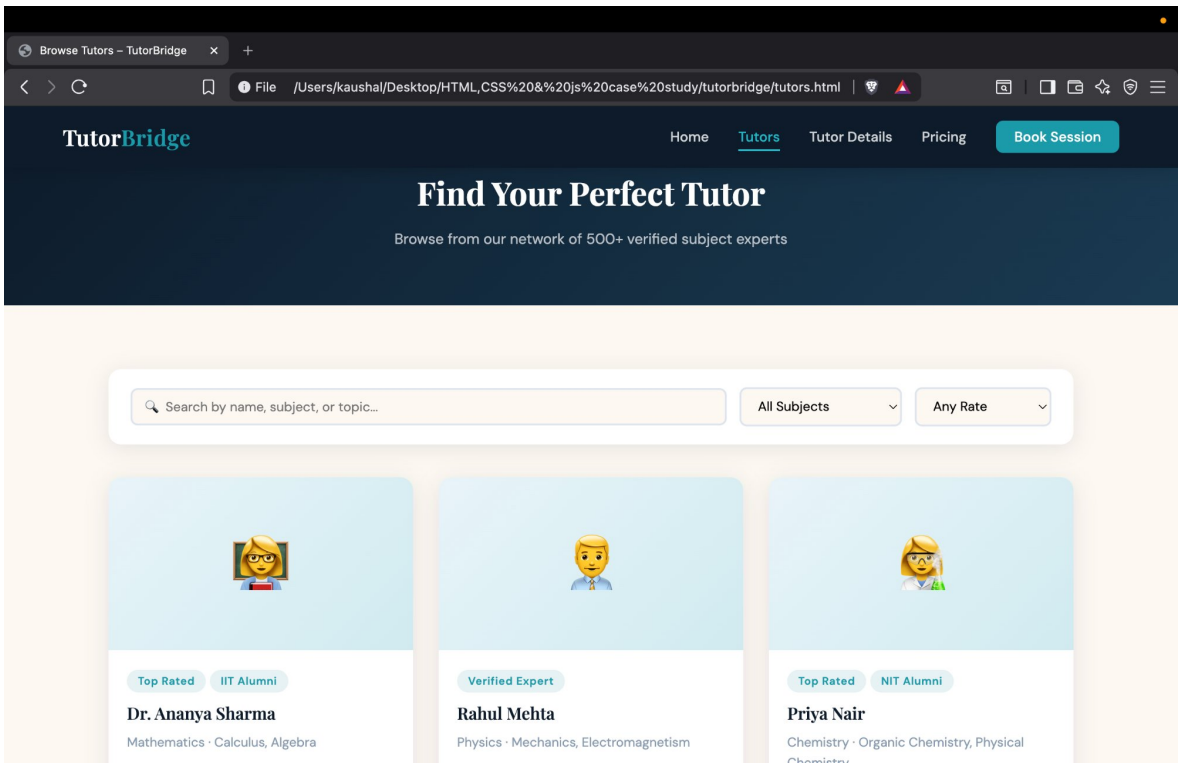
Snapshots

Screenshot 1: Home Page (index.html)



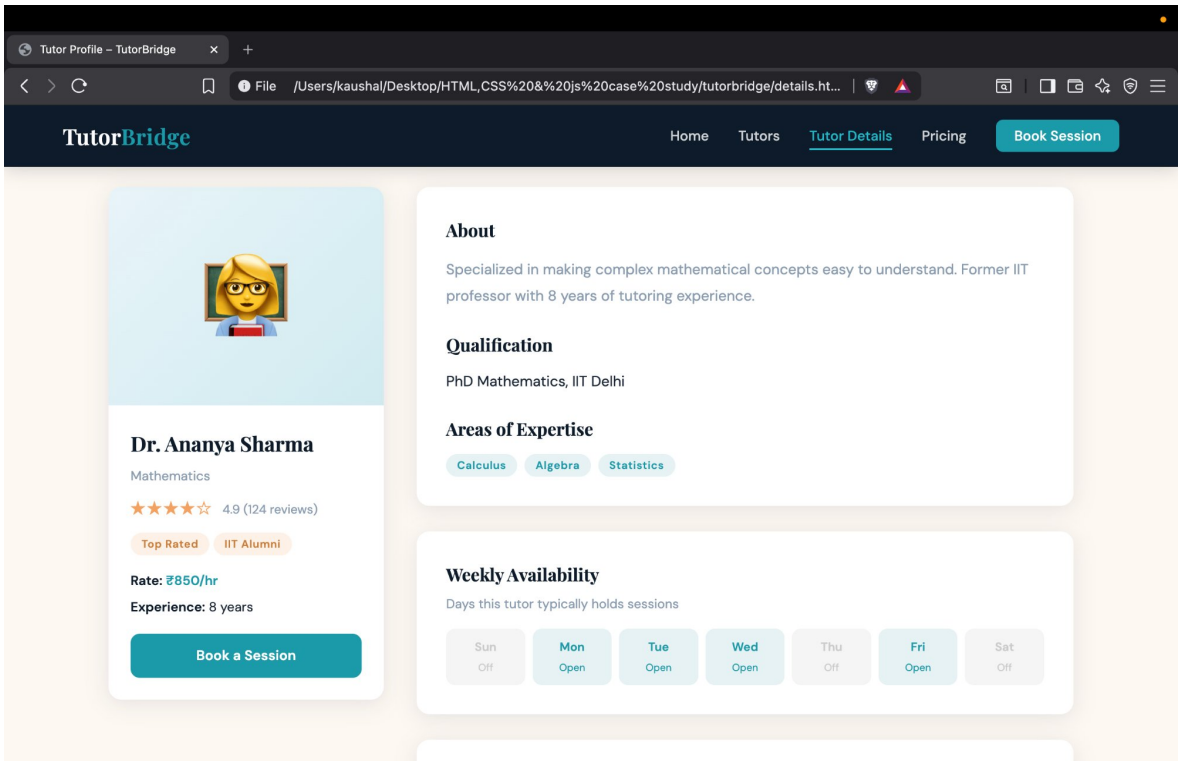
Hero banner with headline, CTA buttons, and stats bar showing 500+ tutors, 12,000+ sessions, and 4.8 average rating.

Screenshot 2: Tutors Page (tutors.html)



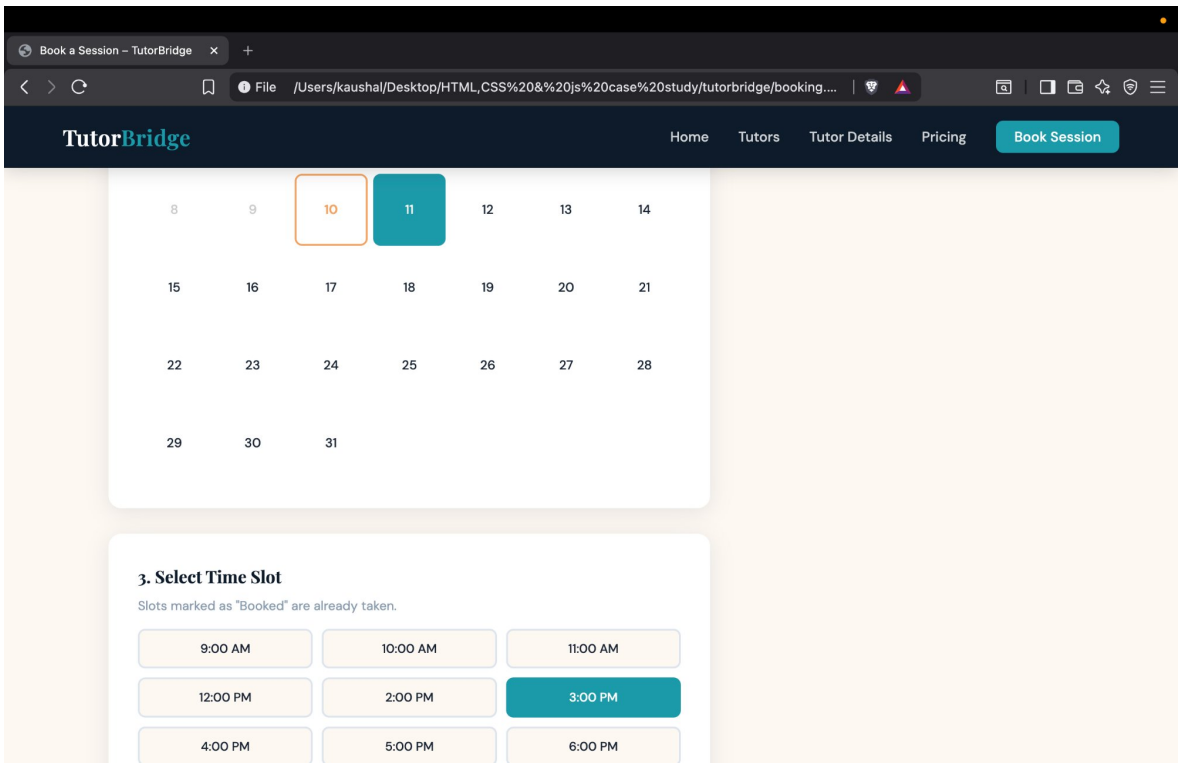
Filter bar with live search and subject/rate dropdowns. CSS Grid layout of tutor cards with badges, ratings, and hourly rates.

Screenshot 3: Tutor Details Page (details.html)



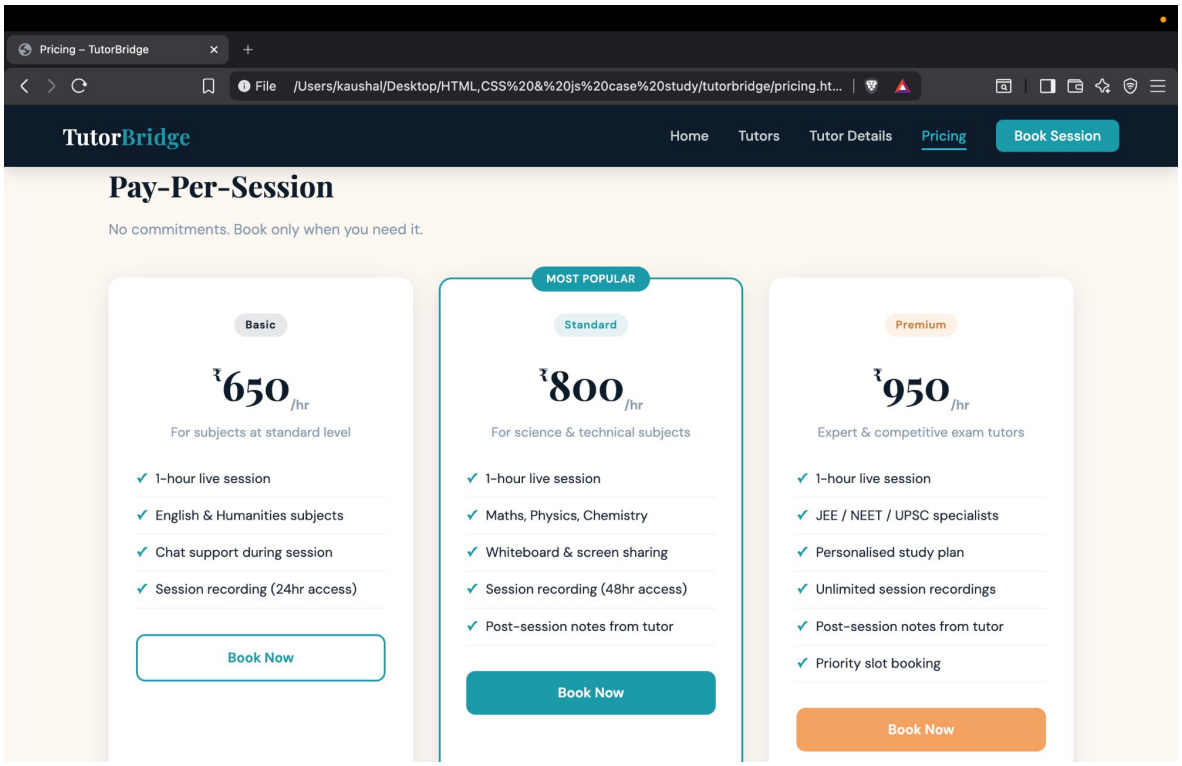
Full tutor profile with sidebar (name, rating, badges, rate, Book Session button) and main area showing bio, qualification, expertise badges, and weekly availability calendar.

Screenshot 4: Booking Page (booking.html)



Interactive monthly calendar with date selected (teal), time slot grid with a selected slot highlighted, and past dates greyed out.

Screenshot 5: Pricing Page (pricing.html)



Three pay-per-session plan cards (Basic, Standard, Premium) with pricing, features list, and Book Now buttons.

6. Results and Conclusion

Findings

- The platform successfully meets all requirements: 5 HTML pages with consistent navigation and footer, external CSS and JS files, no inline styling or scripts.
- CSS Flexbox and Grid work effectively together — Flexbox handles one-dimensional layouts (nav, card footers) while Grid manages two-dimensional layouts (tutor listing, time slots, subject categories).
- Semantic HTML improves readability and structure. Using header, nav, section, article, and footer tags makes the code self-documenting and accessible.
- JavaScript DOM manipulation without any framework demonstrates that vanilla JS is sufficient for functional interactivity including dynamic rendering, filtering, calendar logic, and form handling.
- The booking calendar correctly prevents past date selection and marks pre-booked time slots as unavailable, fulfilling the simulation requirement.
- The live booking summary sidebar updates in real time with tutor name, subject, date, time, and session cost, providing immediate visual feedback to the user.
- The responsive design ensures all pages render properly on mobile devices using media queries and fluid grid layouts.
- Fade-in animations using @keyframes with staggered animation delays create a polished, professional feel without relying on external libraries.

Conclusion

TutorBridge demonstrates a complete, production-quality web application built entirely with core web technologies — HTML5, CSS3, and vanilla JavaScript. The project successfully implements all required features: semantic HTML structure, CSS Flexbox and Grid layouts, star rating visuals, qualification badges, hover effects, fade-in animations, smooth scrolling, input focus styling, and a fully functional JavaScript booking system with calendar logic, booked-slot prevention, dynamic summary display, and booking confirmation.

The modular JavaScript architecture — with a single shared data array and page-detection guards on each init function — ensures clean, maintainable code that scales well. The consistent design language across all five pages, driven by CSS custom properties and a shared stylesheet, demonstrates best practices in front-end development.

This case study reinforces that thoughtful UI design, clean code structure, and well-organised JavaScript logic are the foundation of effective web development — skills that are directly applicable to real-world industry projects.

7. References

1. MDN Web Docs — Official documentation for HTML5, CSS3, and JavaScript. https://developer.mozilla.org

2. CSS-Tricks — Guides on Flexbox and CSS Grid layouts. https://css-tricks.com

3. Google Fonts — Playfair Display and DM Sans typefaces. https://fonts.google.com

4. W3Schools — HTML semantic tags, form elements, and JavaScript DOM reference. https://www.w3schools.com

5. JavaScript.info — In-depth JavaScript tutorials including DOM manipulation, events, and date handling. https://javascript.info

6. Can I Use — Browser compatibility reference for CSS and JavaScript features. https://caniuse.com

7. GitHub — Version control and repository hosting for the project. https://github.com

8. ITM Skills University — B.Tech CSE Semester II Course Guidelines, HTML-5, CSS, JavaScript Case Study Problem Statement, 2025-29.